Unicity Asset Bridging

Bidirectional bridging between Unicity and an external settlement chain

Risto Laanoja

Unicity Labs

July 10, 2026

TL; DR summary

- Bidirectional bridging with **no trusted operator, committee, challenge window, or light client**.
- **Bridge-in**: a permissionless lock-backed mint each recipient verifies for itself; instant for the depositor.
- **Bridge-out**: a succinct validity proof compresses full token-history verification into one constant-cost on-chain check.
- On-chain replay state is a **single accumulator root**, rebuildable by anyone from public events.
- The prover is **disposable**: no authority, fully replaceable; a stall costs at most a fee.
- Chain-independent by construction: a new asset or chain is a new configuration and a new adapter, not new protocol.
- Trust reduces to one assumption: the settlement chain's own consensus.

Full data structures, relation, algorithms: Unicity Yellowpaper Appendix *Asset Bridging*.

Setting

A fungible Unicity token may represent an asset held in custody on an *external chain* (χ), like an account-based EVM/TVM chain. Bridging is **bidirectional**:

- **Bridge-in** (*lock* $\rightarrow$ *mint*): lock the external asset in a settlement contract, the *vault*; mint a Unicity token whose genesis *backing reason* binds it to that lock.
- Now the token can be freely transacted on Unicity, can be split if it is fungible.
- **Bridge-out** / *return* (*burn* $\rightarrow$ *release*): burn the Unicity token; release the matching external asset from the vault.

Design properties

Trust

- No trusted operator, committee, challenge window.
- Return authorized only if verified by an immutable algorithm.

User experience

- Bridge-in to one's own predicate is *almost instant* (no finality wait for minting, finality on source blockchain needed for the first recipient).
- Bridge-out takes some time because of SNARK generation and batching.
- Minimal possible source blockchain fees.
- A stalled prover costs nothing; anyone can complete a return.

On-chain footprint

- Vault is one shared pool (one number)
- Returned asset replay state is **one accumulator root**.
- Constant-cost proof verification, independent of token history and batch size.
- Off-chain state is rebuildable from public source-blockchain events alone.

Proving complexity

- History-dependent token verification is compressed into one succinct proof.
- Proving effort depends on batch size, and number of transactions per token. (sub-linear)

System model and trust base

The vault is an external-chain contract that:

- holds *pooled* custody of one asset a_{A};
- has persistent storage;
- verifies a succinct proof for *constant* cost;
- pins two immutable values:
  - an allow-listed trust-base hash $h_{\mathcal{T}} = H(\mathcal{T})$ — *which* Unicity Service instance is authoritative;
  - a configuration hash h_{cfg} committing the deployment.

A deployment fixes a configuration

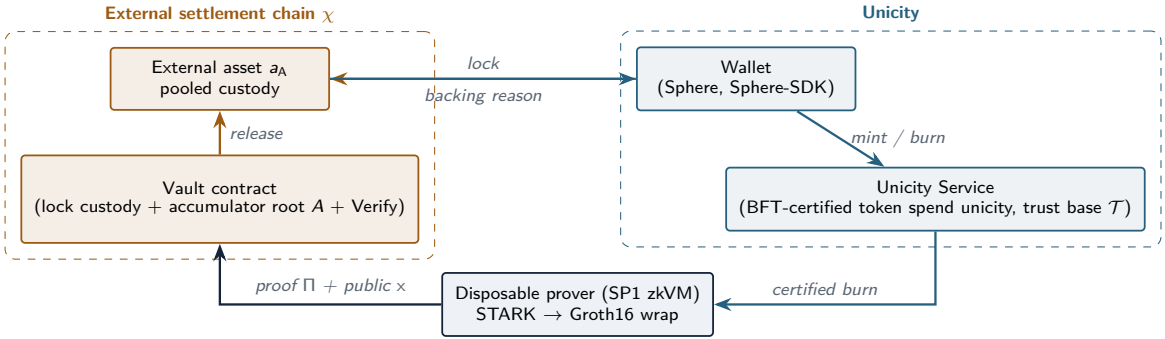
$$\mathcal{C} = (\chi,\ a_{\mathsf{V}},\ a_{\mathsf{A}},\ \mathsf{ty},\ \mathsf{aid},\ d_{\mathsf{lock}},\ d_{\mathsf{nul}},\ t_{\mathsf{reason}}),$$

$$h_{\mathsf{cfg}} = H(\mathsf{CFG\_DOMAIN},\ \chi, a_{\mathsf{V}}, a_{\mathsf{A}}, \mathsf{ty}, \mathsf{aid}, d_{\mathsf{lock}}, d_{\mathsf{nul}}, t_{\mathsf{reason}}).$$

χ external chain id a_{V} vault (contract) address on χ a_{A} external asset (token contract) address ty bridged Unicity token type aid asset identifier read from a token's payload $d_{\mathsf{lock}}, d_{\mathsf{nul}}$ lock-digest / nullifier domain-separation tags t_{reason} CBOR tag of the return reason

The vault derives h_{cfg} *and* its custody asset a_{A} from the same configuration value, so the asset the proof validates against and the asset the vault pays out cannot diverge.

High-level architecture



In vs out

The two directions are *not* symmetric, because the two chains have different finality semantics and different verification costs.

Bridge-in external → Unicity

- External locks have *probabilistic* finality.
- A recipient must confirm the lock is final on χ.
- Realized as a *mint-reason verifier* each recipient re-runs (reading external state).
- Cost of verification is small and client-side; the wait is intrinsic to the external chain.

Bridge-out Unicity → external

- A Unicity burn is BFT-final the instant it is certified; offline-verifiable by VerifyToken.
- The vault has high gas fees: re-running VerifyToken on-chain is prohibitively expensive, grows with number of txs accumulated.
- Realized as a *succinct validity proof* that compresses the history-dependent check off-chain into one constant-cost verification.

Bridge-in has a finality *wait* but trivial verification; bridge-out has instant finality but needs a compact *proof*.

Bridge-in: lock, record, mint

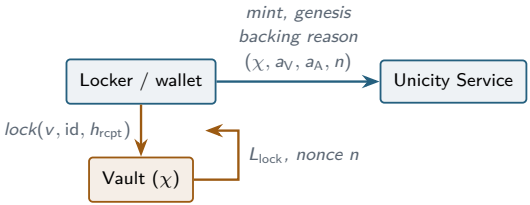
The locker fixes the recipient predicate ν_0' and a salt, determining

$$\mathrm{id} = H(\mathrm{salt}, \alpha), \quad h_{\mathrm{rcpt}} = H(\mathrm{CBOR}(\nu_0')).$$

The vault escrows the asset, assigns a monotone lock nonce n, and emits a *lock event*

$$L_{\mathrm{lock}} = (n,\ a_{\mathrm{src}},\ v,\ \mathrm{id},\ h_{\mathrm{rcpt}}).$$

α: Unicity network id a_{src}: depositor v: locked amount. It stores *only* the *lock digest* of the record $\mathcal{K} = (a_{\mathrm{A}}, \mathrm{ty}, \mathrm{aid}, v, \mathrm{id}, h_{\mathrm{rcpt}})$: $d = H(\mathrm{LOCK\_DOMAIN}, \chi, a_{\mathrm{V}}, n, \mathcal{K})$, $\mathrm{lockDigest}[n] \leftarrow d$ — d is the *only* return-relevant state the vault keeps.



Minting is *permissionless*: the minter key is a public constant derived from id. Authority comes entirely from the backing reason.

Bridge-in: backing verification

Every recipient runs the token-type mint-reason check against the pinned $\mathcal{C}$ and an external-finality oracle $\textsc{LockFinal}_{\chi}, a_{\mathsf{V}}, n$, on D_{mint}, the genesis's mint-reason data:

$\textsc{VerifyBackingReason}(D_{\mathsf{mint}}, \mathcal{T}, \mathcal{C})$ (essentials)

1. decode $(\mathsf{ver}, \chi', a'_{\mathsf{V}}, a'_{\mathsf{A}}, n)$; ensure ver=1 and $(\chi', a'_{\mathsf{V}}, a'_{\mathsf{A}})$ equal the pinned trust anchors;
2. $(\mathsf{ok}, a''_{\mathsf{A}}, v, \mathsf{id}_L, h_{\mathsf{rcpt}}) \leftarrow \textsc{LockFinal}_{\chi'}, a'_{\mathsf{V}}, n$; ensure ok=1, $a''_{\mathsf{A}}{=}a'_{\mathsf{A}}$;
3. ensure $\mathsf{id}_L = H(\mathsf{salt}, \alpha)$ *(lock funds this exact token)*
4. ensure $h_{\mathsf{rcpt}} = H(\mathsf{CBOR}(\nu'))$ *(lock binds this recipient)*
5. ensure locked amount v = genesis value for aid *(no inflation)*

The reason body carries *only* the lock locator and trust anchors. Every security-relevant value — identifier, recipient, amount — is read from the certified genesis and the lock event and checked against each other, never trusted from the reason.

$\textsc{LockFinal}$ is the one step Unicity cannot internalize: a live χ query with K confirmations (a fixed confirmation-depth threshold), *or* a carried inclusion proof against a pinned χ anchor. This is the bridge's only external-chain assumption.

Bridge-in: security

An accepted bridged genesis implies a final lock on χ committing to the same identifier, recipient, and amount. Enumerated guarantees:

- **No mint without a lock** — genesis accepted only against a matching *final* lock.
- **No double mint** — one genesis per id; the lock commits to id, so one lock funds exactly one token.
- **No theft** — h_{rcpt} binds ownership to the named predicate; only the intended recipient can spend.
- **No value inflation** — declared genesis value must equal the locked amount.
- **No rogue contract** — chain, vault, and asset must equal the pinned $\mathcal{C}$.
- **No reorged lock** — the lock must be final on χ.

The wait is enforced by each verifier for itself, not attached to the token as a global condition. The locker minting to a predicate it already controls needs *no* wait — it witnessed its own deposit; a wallet may show such a token to its owner immediately. Only the first party that did *not* witness the lock must run LockFinal.

Bridge-out: setting and return authorization

A Unicity burn is final the instant it is BFT-certified and verifiable offline by VERIFYTOKEN against the trust base $\mathcal{T}$ alone.

To return, the holder forms a *return reason* $\mathcal{R}$ and burns the token with

$$\text{aux}' = b_{\mathcal{R}}, \qquad \nu' = \nu_{\text{burn}}(H(CBOR(\mathcal{R}))),$$

where aux′ is the burn transaction's auxiliary-data field and ν' its next (here: burn) predicate. $\mathcal{R}$ is therefore part of the *certified* burned token, not a separate witness the prover supplies.

Return reason $\mathcal{R}$ (public):

0	version = 1
1–5	$\chi, a_{\mathsf{V}}, a_{\mathsf{A}}$, ty, aid (pinned)
6	a_{r} external recipient
7	v gross = burned value for aid
8–9	$a_{\mathsf{f}}, v_{\mathsf{f}}$ optional relayer fee
10	t_d deadline (gates the fee only)

Every field is public by nature — the external release is a public event. The *private* part of a return is the token's transfer history, which never leaves the witness.

Bridge-out: the nullifier

Replay prevention uses a *nullifier accumulator* — an authenticated set of already-released burns, whose root A the vault stores on-chain.

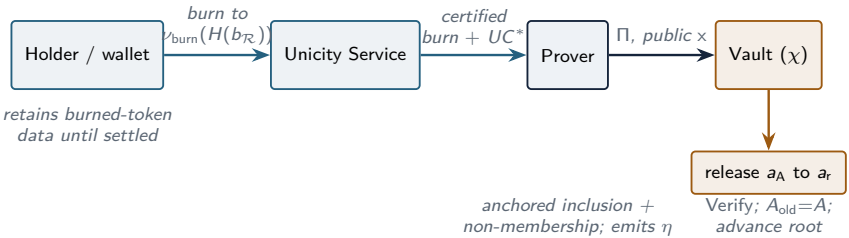
The nullifier is derived from the certified burn leaf. For the final certified transfer (burn transaction) T_{b}, spending state $(\nu_{n-1}, h_{\mathrm{st}\,n-1})$ — predicate and state hash of the token just before the burn — with data field D and data hash $h_{\mathrm{tx\,b}} = H(T_{\mathrm{b}}.D)$:

$$\mathrm{sid}_{\mathrm{b}} = H(\nu_{n-1}, h_{\mathrm{st}\,n-1}), \qquad \mathrm{btid}_{\mathrm{b}} = H(\mathrm{BURN\_DOMAIN}, \mathrm{sid}_{\mathrm{b}}, h_{\mathrm{tx\,b}}),$$

$$\boxed{\eta = H(\mathrm{NUL\_DOMAIN},\ h_{\mathrm{cfg}},\ \mathrm{btid}_{\mathrm{b}})}$$

$\mathrm{sid}_{\mathrm{b}}$ is recorded *at most once* by the Unicity Service (the same mechanism that prevents double-spending), so η is globally unique. The certified $h_{\mathrm{tx\,b}}$ binds it to the exact return reason; h_{cfg} separates deployments so the same burn never collides across independent bridges. A release proves $\eta \notin A$ and inserts it; the vault advances A. On-chain replay state stays **constant** — one root.

Bridge-out: flow



UC^* is a recent Unicity certificate anchoring the burn's inclusion proof (defined next). The prover cannot alter recipient or amount (fixed in the certified $\mathcal{R}$) nor fabricate a release; it can only stall. Any party holding the burned token can reproduce the proof and submit it.

The validity statement

The public statement committed by the proof and checked by the vault is

$$\mathrm{x} = (h_{\mathrm{cfg}},\ h_{\mathcal{T}},\ A_{\mathrm{old}},\ A_{\mathrm{new}},\ \mathrm{rr},\ \mathrm{lr},\ B,\ V),$$

with $A_{\mathrm{old}}/A_{\mathrm{new}}$ the accumulator transition, B the batch size, V the total gross amount, and rr, lr vector commitments to the release leaves and lock references.

For a batch of B burns the proof attests the conjunction:

- **Anchor** — one recent certificate UC^* certifies SMT root h^* under $\mathcal{T}$, and $H(\mathcal{T}) = h_{\mathcal{T}}$.
- **Finality** — each burned token verifies by $\textsc{VerifyToken}$, every leaf included under the shared h^*; the final state is a burn whose reason is $\mathcal{R}$.
- **Value & backing** — each burned amount equals the certified value for aid, tracing — through split value-conservation — to a genesis whose backing references a recorded lock.
- **Destination** — each $\mathcal{R}$ matches $\mathcal{C}$ and yields a public release leaf $(\eta, a_{\mathrm{r}}, v, a_{\mathrm{f}}, v_{\mathrm{f}}, t_d)$.
- **Replay** — each η absent from A_{old}; inserting them yields A_{new}.

The batch proving relation $\mathfrak{R}(\mathrm{x}, \mathrm{w})$

Witness w: the config $\mathcal{C}$, trust base $\mathcal{T}$, anchor UC^*, and per burn i the source token $\mathcal{L}_i$, its anchored inclusion proofs, the backing lock record, and a non-membership witness w_i.

$H_{\mathrm{cfg}}(\mathcal{C}) = \mathrm{x}.h_{\mathrm{cfg}}$; $H(\mathcal{T}) = \mathrm{x}.h_{\mathcal{T}}$; verify UC^* once, $h^* \leftarrow UC^*.IR.h$; $A \leftarrow \mathrm{x}.A_{\mathrm{old}}$.

for each $i = 1 \ldots B$:

1. VerifyToken$\mathcal{L}_i, \mathcal{T}$ in *anchored* mode against h^*; recompute $\mathrm{sid}_{\mathrm{b},i}, h_{\mathrm{tx\,b},i}$ from the final certified burn.
2. decode $\mathcal{R}_i$ from the burn aux; ensure fields 1–5 equal $\mathcal{C}$.
3. value v_i from the genesis for aid; ensure $\mathcal{R}_i.v = v_i$ and $\mathcal{R}_i.v_{\mathrm{f}} \leq v_i$.
4. backing: recompute $d_i = H(\mathrm{LOCK\_DOMAIN}, \chi, a_{\mathrm{V}}, n_i, \mathcal{K}_i)$; ensure $\mathcal{K}_i$ binds this id, recipient, aid, v_i.
5. replay: η_i from the burn; ensure VerifyNonMember(A, η_i, w_i); $A \leftarrow \mathrm{Insert}(A, \eta_i, w_i)$.

final: $A = \mathrm{x}.A_{\mathrm{new}}$, $\sum v_i = \mathrm{x}.V$, Commit(leaves) $= \mathrm{x.rr}$, Commit(lockrefs) $= \mathrm{x.lr}$.

Anchored inclusion: one certificate per batch

Because the Unicity SMT is **add-only**, every state a token ever occupied remains provable under any later root. So one recent certificate anchors entire token histories.

- Fix an anchor certificate UC^* with root $h^* = UC^*.IR.h$.
- For each certified transaction leaf $L = (\text{sid}, h_{\text{tx}})$, carry an SMT inclusion certificate C^{inc} of L against h^*.
- The relation verifies UC^* *once* via VerifyUnicityCert, then each L by a hash-path check $\text{VerifyMember}(h^*, L, C^{\text{inc}})$.

This replaces m BFT-certificate verifications (one per transaction leaf L across all burned tokens' histories) by **one** certificate plus m hash-path checks — the dominant saving inside the circuit.

Anchored inclusion omits the original request-validation time, so the relation is defined only for *time-independent* predicates (signature/burn/split). Time-dependent predicates (timelocks/HTLCs) require carrying authenticated validation time — future scope.

Trustless setup

No party — including the deployer — can move funds except by presenting a mathematically valid proof against an *immutable* relation.

- **Immutable** vk — the verification key is fixed at deployment; the relation cannot be silently swapped. Changing it requires a *new* vault (§upgrades).
- **Prover holds no authority** — recipient and amount are fixed inside the certified $\mathcal{R}$; the prover can only stall, never steal or forge.
- **Each recipient is its own verifier** — bridge-in acceptance is a check every recipient re-runs; no bridge party attests to anything.
- **No committee, no challenge window** — the vault releases funds *only* against a valid succinct proof — there is nothing to optimistically trust and later dispute.
- **Authenticity by soundness** — $\mathfrak{R}$ embeds $\textsc{VerifyToken}$ against the pinned $\mathcal{T}$; a forged, malformed, or non-final burn is unrepresentable under proof-system knowledge-soundness.

The *only* external assumption is the settlement chain's own consensus (bridge-in finality) — nothing else is trusted.

The disposable prover

The prover is a pure function of *public* inputs plus the holder-retained burned-token data. It has no keys, no custody, no privileged state — so it is fully replaceable.

Liveness argument. From the on-chain root and the public Released event stream, *any* party can:

1. rebuild the off-chain accumulator to A;
2. refetch anchored inclusion proofs from the Unicity Service;
3. reproduce the proof and submit it.

The only residual requirement is availability of the burned-token data, which the holder retains until settlement.

Consequences

- A stalled or absent relayer costs at most the fee v_f: after the deadline the recipient receives the full amount, and anyone — including the recipient — may submit.
- The prover can be a self-hosted, single-machine, sequential process. No Docker, no GPU, no proving network required for correctness.
- A defective prover cannot damage the vault; at worst it produces no proof.

The on-chain contract set

Vault (one per deployment) — persistent state:

- vk, h_{cfg}, custody asset a_{A};
- allow-list of trust-base hashes (rotatable under timelock);
- bridge-in map lockDigest[·];
- accumulator root A (init $A_{\emptyset}$).

Entry points

- lock$(v, \mathsf{id}, h_{\mathsf{rcpt}})$ — escrow, assign n, store d, emit L_{lock}.
- FulfilBatch$(\mathsf{x}, \Pi, \langle \ell_i \rangle, \langle (n_j, d_j) \rangle)$ — ℓ_i a release leaf $(\eta, a_{\mathsf{r}}, v, a_{\mathsf{f}}, v_{\mathsf{f}}, t_d)$, (n_j, d_j) the referenced lock nonces/digests.
- setTrustBaseAllowed (timelocked, governance).

SP1Verifier — one global bn254 Groth16 verifier, shared by all vaults.

FulfilBatch (settlement):

1. Verify$(\mathsf{vk}, \mathsf{x}, \Pi) = 1$;
2. $\mathsf{x}.h_{\mathsf{cfg}} = h_{\mathsf{cfg}}$, $\mathsf{x}.h_{\mathcal{T}} \in$ allow-list;
3. $\mathsf{x}.A_{\mathsf{old}} = A$ *(the entire replay guard)*
4. commitments rr, lr re-checked; each lockDigest$[n_j] = d_j$;
5. $A \leftarrow \mathsf{x}.A_{\mathsf{new}}$ *(one root update)*
6. pay each leaf; fee only if now $\leq t_d$ and $a_{\mathsf{f}} \neq \perp$;
7. emit Released$(\eta, \dots)$, BatchFulfilled$(\dots)$.

No external query, no light client: the proof self-certifies its anchor under the allow-listed $\mathcal{T}$; backing is confirmed against locally recorded digests. The vault tracks *no* Unicity roots.

On-chain data is minimal

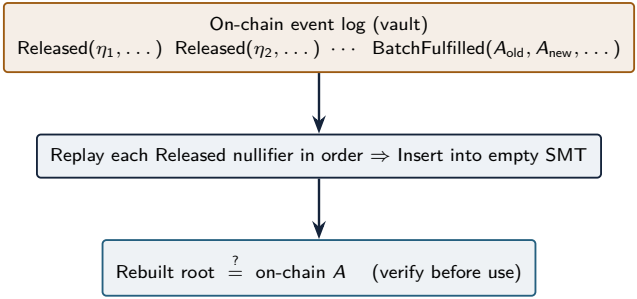
The vault keeps only what is needed to be safe; *nothing* that grows with Unicity history.

State	Size	Grows with
vk, h_{cfg}, a_{A}	constant	— (immutable)
Trust-base allow-list	tiny	validator-set epochs
Accumulator root A	**one hash**	— (constant)
lockDigest[n]	one hash / deposit	bridge-in deposits

- Replay prevention for the entire return history is a **single root**.
- No per-transition certificates, no Unicity roots, no token histories are ever stored on-chain.
- The split (decision): keccak256/ABI for vault-recomputed commitments (h_{cfg}, d, rr, lr); SHA-256/CBOR for Unicity-internal values (η, accumulator, $h_{\mathcal{T}}$) that the vault never recomputes.

Nullifier tree: reconstruct from on-chain data alone

The accumulator is a depth-256 sparse Merkle tree keyed by η, leaf value `0x01`. It is **deterministically rebuildable by anybody** from public events, based on data availability guarantee of the source blockchain.



- Released emits each nullifier; BatchFulfilled emits each root transition — so the off-chain accumulator is rebuildable from public data alone and *verified against the on-chain A* before use.
- The service additionally re-verifies the rebuilt root equals the vault's on-chain root (SYNCED gate) before building a new batch.

Settlement properties

- **No external query, no light client** — the proof self-certifies its anchor; backing checked against local digests; the vault tracks no Unicity roots.
- **Fee optionality never strands funds** — t_d and a_f gate the fee *only*. After the deadline, or when $a_\mathsf{f} = \bot$, the recipient gets the full amount; anyone may submit.
- **Batch atomicity** — all B transfers settle together; a reverting recipient reverts the batch, but A is unchanged and no nullifier is consumed — a liveness, not a safety, concern. A deployment expecting blockable recipients may credit withdrawable balances instead.
- **Reconstructability** — Released + BatchFulfilled make the accumulator rebuildable from public data.

Implementation note. On EVM/TVM chains a low-level asset transfer may not receive the caller's full remaining gas/energy after an expensive proof verification in the same transaction; the vault budgets an explicit sufficient fee per transfer rather than relying on default forwarding.

Economics of batching

The public statement and settlement procedure are identical for every batch size B; only the prover changes. The ≈constant proof verification is shared across the whole batch, so per-return cost falls as $1/B$.

$$\textsc{FulfilBatch}(B) \approx 267{,}779 + 19{,}347 \cdot B \text{ (energy)}$$

B	per-return energy	per-return \$*
1	287,126	∼9.0
10	46,125	∼1.5
100	22,025	∼0.7
∞	19,347	∼0.6

*Tron mainnet, measured Nile energy, 210 sun/energy, \$0.15/TRX (burn model).

Cost structure

- Proof verify (∼218k energy) is *per batch* ⇒ amortizes $1/B$.
- Only the ∼19k/return transfer + leaf checks are truly per-return.
- Off-chain proving is one Groth16 proof per batch — chain-agnostic; amortizes $1/B$ too.
- Staking energy (Tron) turns the recurring fee into refundable capital ⇒ marginal on-chain cost → 0.

The relayer fronts energy and is reimbursed via v_f; the vault enforces $v_f \leq v$ and pays the fee only before t_d.

Integrating the next asset and the next chain

The relation $\mathfrak{R}$ and the vault are **chain-independent**; a new deployment is a new configuration, not new protocol.

- **New asset, same chain** — fix a fresh $\mathcal{C}$ (new a_{A}, ty, aid), deploy a vault with the resulting h_{cfg}, publish h_{cfg} + allowed $h_{\mathcal{T}}$. The circuit is unchanged.
- **New chain** — select chain-specific parameters — address width, the LockFinal realization (K confirmations or a pinned anchor), the vector-commitment primitive — and redeploy. The settlement chain is abstracted as "contract with storage + verifier".
- **Wallet/SDK side** — a neutral `@unicitylabs/bridge-core` defines `BridgeSourceAdapter` / `ChainWallet` / `ChainClient`; a chain is added as a *new adapter* producing opaque `Step[]` (e.g. ERC-20 `approve+lock` vs. single-signature EIP-3009). The wallet's state machine is identical.

The manifest is a discriminated union keyed on chain family with a CAIP-2 chain reference; adding a family is additive, never a fork. The generic justification payload carries the full h_{cfg} — identifying vault *and* complete trusted configuration in one field.

Implementation: software components

Monorepo

`github.com/unicitylabs/unicity-bridge`

- `protocol/` — the normative byte contract (`interop.md`) + conformance vectors every stack must reproduce.
- `contracts/tron/` — vault & lock TVM contracts (`UnicityBridgeVault`, `UnicityLock`, SP1-verifier binding), Hardhat tests.
- `packages/bridge-core` — chain-neutral TS interfaces (`BridgeSourceAdapter`/`ChainWallet`/`ChainClient`).
- `packages/bridge-plugin-tron-usdt` — first chain plugin: Tron USDT verifier, wallet adapter, CLI/demo.
- `prover/` — Rust/SP1 return-prover workspace: `core` (no_std byte contract), `guest` (SP1 circuit), `host`, `sdk-ext` (bridge-only extensions over the token SDK), `service` (return service).
- `deployments/` — frozen deployment metadata.

Wallet side

- `sphere` @ `feat/unicity-bridge` — the wallet UI/UX (display, bridge-in/out flows).
- `sphere-sdk` @ `feat/unicity-bridge` — façade + manifest wiring (previous/next frame).

The Rust SDK

`github.com/unicitynetwork/state-transition-sdk-rust`

- `no_std`-first and zkVM-ready *by design* — it is what the SP1 guest links against to re-verify full token history *inside the circuit*.
- Wallets/services that don't prove keep using the existing `state-transition-sdk-js`; the Rust SDK was developed specifically to make in-circuit verification possible.

Sphere / SDK integration

Boundary (hard rule). Sphere holds no bridge logic: all derivations and chain specifics are provided by the plugin; Sphere imports a *façade* + a *manifest* and renders UI.

`buildBridgeInPlan` → {tokenId, recipientCommitment, approveTx, lockTx}
`buildBridgeBackPlan` → {burnTransfer, preview, witnessRequest}
`previewReturn`, `decodeBridgeBackReason` (read-only UI).

The SDK exposes `bridges: BridgeManifest[]`; plugins are built via the façade and their justification verifiers forwarded into the token engine — no core engine change.

User-facing flows

- **Display**: bridged balances carry a "bridged (USDT · Tron)" badge; unknown/failed re-verify → "unverified", excluded from spendable; "confirming (N blocks)" during finality.
- **Bridge in**: `approve` (one-time) → `lock` → mint at `confirmations:0` (self-trust) ⇒ token spendable immediately; pending-lock recovery resumes an interrupted mint.
- **Bridge out**: pick a returnable balance, dest + amount; sign the Unicity burn; persist the burned blob (recovery-critical); poll the return service *and* watch `Released`$\{\eta\}$; self-settle after the deadline.

A "returnability" flag (time-independent-predicate lineage) gates "Bridge out". Cross-stack conformance vectors keep Solidity, TypeScript, and Rust byte-identical.

Security properties (summary)

- **Solvency** — every bridged token is backed by an equal lock; split verification bounds children by their parent; the backing check ties each burn to a recorded lock digest. Hence $\sum(\text{released}) \leq \sum(\text{locked})$ — custody cannot be over-drawn.
- **Replay-freeness** — nullifiers are globally unique; each release proves non-membership and inserts; the vault requires $A_{\text{old}} = A$. A resubmitted proof fails (root advanced); a fresh proof cannot show non-membership of an inserted nullifier.
- **Authenticity & finality** — $\mathfrak{R}$ embeds $\textsc{VerifyToken}$ against the pinned $\mathcal{T}$; by knowledge-soundness an accepting proof implies a genuine, BFT-final burn whose reason is exactly $\mathcal{R}$.
- **Liveness, data availability** — the prover holds no authority and is replaceable; any party rebuilds the accumulator from public data (the source blockchain) and reproduces a proof. Residual requirement: availability of the burned-token data, retained by the holder.